

ONE PROBLEM, START TO FINISH

Transfer Learning a GNN across cities with different graphs

Same forecasting task, but each city is a graph of a different size:

Delhi

40 nodes

Kolkata

10 nodes

Guwahati

4 nodes

The central question of this entire deck:

*"How can ONE trained GNN move from a 40-node city to a 4-node city,
when their graphs do not even have the same shape?"*

One task, many cities — but the graphs differ

- Each city's monitoring network is a graph: a station is a node, nearby stations are linked.
- The forecasting task is identical everywhere: predict each station's PM2.5 a few hours ahead.
- But the graphs are wildly different in size: Delhi 40 nodes, Kolkata 10, Guwahati just 4.
- And the small cities are exactly the data-scarce ones where we most need help.
- Transfer learning across cities is well studied for time series, but usually assumes
- matched structure. Here the structure itself changes — that is the crux of the problem.

Goal: train where data is plentiful (Delhi), then reuse that model where data is scarce (Guwahati).

Literature: Cross-city TE — Wei, Zheng & Yang (KDD 2016), RegionTrans — Wang et al. (IJCAI 2019), MetaST — Yao et al. (WWW 2019).

Why an ordinary GNN cannot make the jump

- A standard 'transductive' GNN bakes the specific graph into its own parameters.
- Example: a learned adjacency matrix has shape $|V| \times |V|$ — tied to the node count.
- Train it on Delhi and that matrix is 40 x 40. It only fits a 40-node graph.
- Try to load it onto Guwahati's 4 nodes and you hit a hard parameter-shape mismatch.
- The weights literally do not line up — the model will not even run, let alone transfer.
- Call this failure mode F-i: any weight whose shape depends on $|V|$ blocks transfer.

Literature: The obstacle is structural, not statistical; the model's shape is welded to one city's size. — Rippe & Weiling (ICLR 2017), adaptive adjacency — Wu et al., MTGNN (KDD 2020).

Make the GNN INDUCTIVE: the graph is an input

Move the graph out of the parameters and into the forward pass.

X Transductive

weights = { W1, W2, ..., A(|V| $\times$ |V|) }

the graph is INSIDE the weights

trained on 40 nodes -> stuck at 40

OK Inductive (ours)

weights = { W1, a_src, a_dst, ... }

forward(x, edge_index, edge_weight)

the graph is fed in as an INPUT

Literature: Inductive GNNs learn rules for ONE node and ONE edge — never for a fixed list of nodes. GraphSAGE — Hamilton, Ying & Leskovec (NeurIPS 2017); GAT — Veličković et al. (ICLR 2018).

Not one parameter depends on the node count

Component	What it learns	Shape	Depends on $ V $?
Temporal conv	how a node evolves in time	$[F, 64, 3]$	No
GAT W	how to project one node	$[64, 64]$	No
GAT $a_{\text{src}} / a_{\text{dst}}$	how to score one edge end	$[64]$	No
Linear head	per-node regression	$[64, 1]$	No

- Every weight describes a single node or a single edge — a local rule, reused everywhere.
- So the same ~25k-parameter encoder runs on 40, 10, or 4 nodes with zero shape change.

Literature: *Local message-passing rules — GraphSAGE (NeurIPS 2017), GAT (ICLR 2018); dilated temporal conv — TCN, Bai et al. (2018).*

Inductive transfer is theoretically grounded

- It is not just an engineering hack — there is theory for transferring across graph sizes.
- Graphon theory: graphs of different sizes drawn from the same underlying 'graph process'
- are limits of one object; a GNN trained on one converges to the same behaviour on another.
- Spectral view: GNN filters act on the graph spectrum, which is stable as the graph grows,
- so the learned operator stays meaningful on a differently sized graph.
- Our three cities are all k-NN graphs of CPCB stations in one climatic region —
- different realisations of a similar spatial process. Exactly the regime the theory covers.

Same trained operator, graphs of different size, provably similar behaviour — that is what licenses the transfer.

Literature: Graphon transferability — Ruiz, Chamon & Ibarra (NeurIPS 2020), spectral transferability — LeVe et al. (JMLR 2021).

The inductive encoder we carry between cities

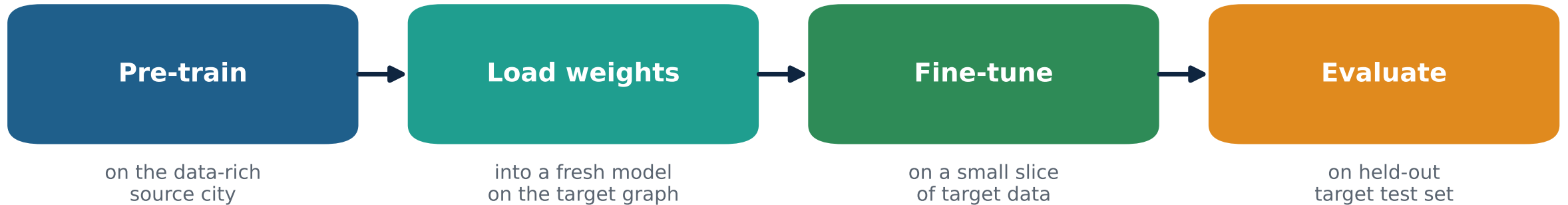
A time-space-time stack — every block is $|V|$ -free, so the whole encoder transfers:



- GAT blocks pass weighted messages between neighbouring stations (the spatial reasoning).
- TCN blocks read each station's recent trend (the temporal reasoning).
- All weights are shared across nodes — exactly why this checkpoint loads onto any city.

Literature: *ST sandwich* — STGCN, Yu et al. (IJCAI 2018) & Graph WaveNet, Wu et al. (IJCAI 2019); *attention* — GAT (ICLR 2018).

Inductive GNN-TL in four moves



- The inductive design makes step 2 trivial: no surgery, no reshaping, no per-city rebuild.
- Only a little target data is needed because we start from a strong, pre-trained model.
- The next slides open up each step.

Pre-train on the data-rich source city

- Build the source city's k-NN graph from station coordinates (edges + distance weights).
- Slice the history into sliding windows: 24 h in -> PM2.5 a few hours ahead, per station.
- Train the full inductive encoder on the source until it forecasts the source well.
- The weights now absorb general structure: daily cycles, pollution episodes, how
- neighbouring stations co-vary along wind corridors.
- Save this checkpoint — it is the knowledge we will carry to the target.

Pre-training is where the transferable knowledge is created; the source city is the 'teacher'.

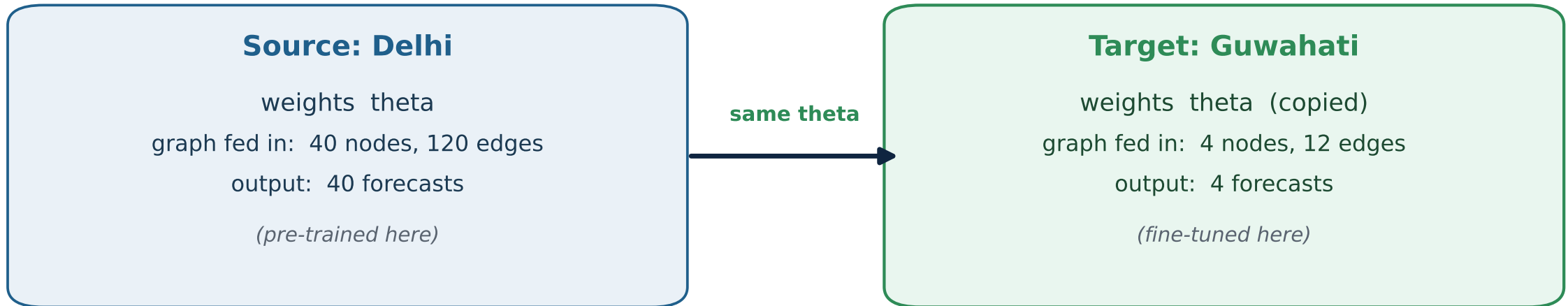
Literature: GNN pre-train then adapt — Hu et al. (ICLR 2020), air quality GNN with wind-aware edges — PM2.5-GNN, Wang et al. (SIGSPATIAL 2020).

The transfer itself, step by step

- 1 Instantiate**
Create a fresh encoder on the TARGET graph (its own edges, its own node count).
- 2 Load weights**
Copy the source checkpoint in. No shape error — nothing depended on $|V|$.
- 3 Freeze low layer**
Briefly lock the bottom temporal block so a tiny target gradient can't wreck it.
- 4 Fine-tune**
Train on $d\%$ of target data at a small learning rate; unfreeze after a warm-up.
- 5 Best-by-val**
Keep the checkpoint that scores best on a target validation split.

Literature: *Freeze-low / fine-tune-high* — Yosinski et al. (NeurIPS 2014); *frozen-layer FT for PM2.5* — Yadav et al. (Env. Modelling & Software 2024).

Worked example: Delhi (40) -> Guwahati (4)



- The ONLY thing that changes is the graph handed to the forward pass (edges + weights).
- The 40x10 size gap needs no architecture change — the same θ drives both.
- Fine-tuning then specialises θ for Guwahati's local dynamics.

A 3-way check, run in every setting

Zero-shot

source weights,
NO target training
= does it generalise
out of the box?

Scratch

random weights,
target data only
= what target data
alone achieves?

Transfer

source weights
+ target fine-tune
= the method
we propose

Transfer is declared REAL only if $\text{Transfer} > \text{Scratch}$ AND $\text{Transfer} > \text{Zero-shot}$

- The scratch control rules out 'the target data alone was already enough'.
- Most cross-city papers skip this control; reporting all three makes the claim falsifiable.

Distribution shift, and the adversarial fix

- Inductivity solves the SHAPE problem. A second problem remains: cities differ statistically.
- Delhi's pollution regime is not Guwahati's, so the copied features can be a poor fit.
- Adversarial domain adaptation aligns them: pool each graph to a fixed-size embedding,
- then add a referee that guesses the city and a gradient-reversal layer that fools it.
- The encoder is pushed toward city-invariant features that are still PM2.5-predictive.
- Mean+max pooling keeps the embedding size-invariant — so one referee compares 40 vs 4 nodes.
- Then a short fine-tune (referee off) specialises for the target, as before.

Phase 1 transfers the weights; this adversarial phase also aligns the feature distribution before fine-tuning

Literature: DAMN / gradient reversal — Gahm & Lempitsky (ICML 2015), Gahm et al. (JMLR 2016), size-invariant readout — GIN, Xu et al. (ICLR 2019).

The cross-city transfer landscape

Cross-city ST transfer

- RegionTrans — Wang et al. (IJCAI 2019)
- MetaST — Yao et al. (WWW 2019)
- ST-GFSL — Lu et al. (KDD 2022)
- CrossTReS — Jin et al. (KDD 2022)
- TransGTR — Jin et al. (KDD 2023)

Graph domain adaptation

- DASTNet — Tang et al. (CIKM 2022)
- UDA-GCN — Wu et al. (WWW 2020)
- EGI — Zhu et al. (NeurIPS 2021)
- Graph DA survey — Shi et al. (2024)
- PM2.5-GNN — Wang et al. (SIGSPATIAL 2020)

Our angle: most assume similar structure; we transfer across an order-of-magnitude size gap.

How we transfer a GNN across cities

- 1 The barrier is structural.**
Different cities = different graph sizes; $|V|$ -tied weights can't move.
- 2 Make the GNN inductive.**
Put the graph in the forward pass, not in the parameters.
- 3 Then one model fits any city.**
No weight depends on node count, so 40 $\rightarrow$ 4 just works.
- 4 Transfer = pre-train + fine-tune.**
Teach on the data-rich source, adapt on a little target data.
- 5 Align distributions too.**
Adversarial domain adaptation handles cross-city statistical shift.
- 6 Verify honestly.**
Real transfer must beat both zero-shot and from-scratch.

Key references:

GraphSAGE (NeurIPS 2017) · GAT (ICLR 2018) · Graphon transfer, Ruiz et al. (NeurIPS 2020) · Spectral transfer, Levie et al. (JMLR 2021)
GNN pre-training, Hu et al. (ICLR 2020) · Yosinski et al. (NeurIPS 2014) · DANN, Ganin et al. (ICML 2015 / JMLR 2016) · DASTNet (CIKM 2022)